Observabilidade e SRE — Venus

Relatório de medição — 30/10/2026

Projeto interdisciplinar de Inteligência Artificial · Venus · ExpoTech, 06/11/2026

Fonte do requisito: *Solicitações por Disciplina*, 14/05/2026, página 11 (Inteligência Artificial, "Mínimo para o 7,0").

Sumário

Resumo executivo 3

1. Escopo, fontes e como ler os números 3

2. Metodologia e definições 3

3. Resultados medidos 4

3.1 Tráfego real do mobile (produção) . . . 4

3.2 Latência por agente e entre agentes . . . 5

3.3 Índice de erros . . . 6

3.4 Demo controlada com o grafo real . . . 6

4. Custo, custo por resolução e ROI 7

5. Projeção para 100 e 1.000 usuários semanais 7

6. Premissas justificadas 8

6.1 Preço por token . . . 8

6.2 Valor por resolução . . . 9

6.3 Definição de resolução e limites . . . 9

7. SLOs e SRE 9

8. Cruzamento /v1/metricas × Langfuse × middleware 10

9. Arquitetura de alto nível 10

10. Evidências do Langfuse 11

11. Como reproduzir cada número 12

12. Limitações 13

13. Validação automatizada 13

14. Pendências que só o aluno resolve 14

Apêndice. Piloto de 19/10/2026 14

Resumo executivo

1. **Erros e disponibilidade (medido em produção):** em 143 requisições ao `POST /v1/chat` entre 13 e 30/10/2026 não houve nenhum HTTP 5xx (disponibilidade de 100%, amostra pequena) e nenhum turno do grafo terminou em erro (0 de 139). Falhas técnicas que voltam como HTTP 200 só são medidas depois do deploy do coletor.
2. **Latência (medido em produção):** p50 de 5,2 s e p95 de 29,3 s por turno (Langfuse); o tempo está dentro dos agentes (especialistas de 10 a 19 s), e apenas 0,57 s por turno (14%) fica entre os agentes. A meta de p95 < 20 s **não foi atingida**.
3. **Custo (estimado com preço pago de referência):** cada turno consome em média 15,4 mil tokens de entrada e custa US$ 0,0032. Com o ritmo médio medido (8,2 turnos por usuário por semana), 100 usuários semanais custam cerca de **US$ 11 por mês** e 1.000 usuários cerca de **US$ 112 por mês** (faixa de US$ 3 a US$ 50 e de US$ 28 a US$ 495, conforme o ritmo de uso).
4. **Custo por resolução e ROI:** com 67,9% de turnos resolvidos na demo, cada resolução custa **US$ 0,0047**; com o valor conservador de US$ 0,50 por resolução o ROI é de **106×** (395× a US$ 1,84 e 2.902× a US$ 13,50). O ROI depende quase só do valor assumido por resolução, que não foi medido.
5. **Achado de SRE:** 73% das chamadas ao `gpt-oss-120b` (Groq, plano gratuito) falham e o fallback absorve sem o usuário ver, mas isso custa latência; e, com a rede instável desta máquina, o Langfuse perdeu até 7% dos traces enquanto o coletor da API registrou 100% dos turnos.

1. Escopo, fontes e como ler os números

O requisito pede: custo estimado para 100 e 1.000 usuários semanais, latência entre agentes e tempo total de resposta, índice de erros, custo/retorno (ROI), custo por resolução e um desenho de arquitetura de alto nível. A escola não paga API de IA generativa, então todo custo aqui é **"quanto custaria"** com preço pago de referência, não cobrança.

Cada número do relatório tem um rótulo:

Rótulo	Significa	Fontes
Medido (produção)	Tráfego real do app mobile, 7 usuários, 14 a 30/10/2026	Langfuse (139 turnos) e `metricas_requisicoes` do Mongo de produção, lido só para contagens (143 requisições)
Medido (demo)	Roteiro controlado de 28 turnos com o grafo real, 4 usuários `demo-*`, rodadas de 30/10/2026	Coletor da API (`/v1/metricas`) e Langfuse
Extrapolado	Projeção por fórmula explícita a partir de números medidos	`scripts/observabilidade/numeros_do_relatorio.py`

Os 7 usuários de produção são os do período de testes do app (um deles responde por 63% dos turnos: 88 de 139). A fotografia de produção foi **congelada em 30/10/2026 às 14:58 UTC** para os números não mudarem enquanto o relatório é escrito.

2. Metodologia e definições

Medida	Origem	Armazenamento
Tempo total do turno	Cronômetro em volta do `ainvoke` em `chat.py`	Mongo `metricas_turnos`
Tempo por agente	`ColetorDoTurno`, callback LangChain, só nós do grafo (sem os nós internos `model` e `tools` do agente ReAct)	`metricas_turnos.nos_ms`
Tokens de entrada e saída	`usage_metadata` de cada chamada de LLM	`metricas_turnos`
Erro do turno	Exceção ou timeout no grafo (HTTP 502 ou 504)	`metricas_turnos.erro`
Falha técnica	Especialista que caiu e respondeu 200 com um pedido de desculpas, conferido **no turno atual**	`metricas_turnos.falha_tecnica`
Status e duração HTTP	Middleware da API	Mongo `metricas_requisicoes`
Spans, tokens e latência por chamada	Langfuse (API v2 de observações)	Langfuse Cloud

- **Turno:** mensagem que chegou ao grafo. Autenticação e limite de taxa (401, 422, 429) não são turnos.
- **Resolvido:** sem erro, com resposta final, juiz não reprovou e nenhum guardrail bloqueou entrada ou saída. A definição não mede satisfação nem correção factual (ver seção 6.3).
- **Índice de erros em três camadas:** (1) HTTP 5xx; (2) turno que terminou em exceção ou timeout; (3) **falha técnica mascarada**, que sai com HTTP 200 e não aparece nas duas primeiras.
- **Latência entre agentes:** do tempo total do turno, o que não está dentro de nenhum agente (decisões entre nós e custo do próprio LangGraph).
- **Custo estimado:** tokens medidos × preço de referência pago (seção 6.1).
- **ROI:** `(resoluções × valor por resolução − custo) ÷ custo`.
- **Projeção:** usuários × turnos por usuário por semana × custo por turno, em escala linear. Não inclui infraestrutura (ECS, Mongo, Postgres, Qdrant).

3. Resultados medidos

3.1 Tráfego real do mobile (produção)

Janela: 14/10 a 30/10/2026 (corte em 30/10, 14:58 UTC). **Fonte:** Langfuse, API v2, sem ler conteúdo de conversas.

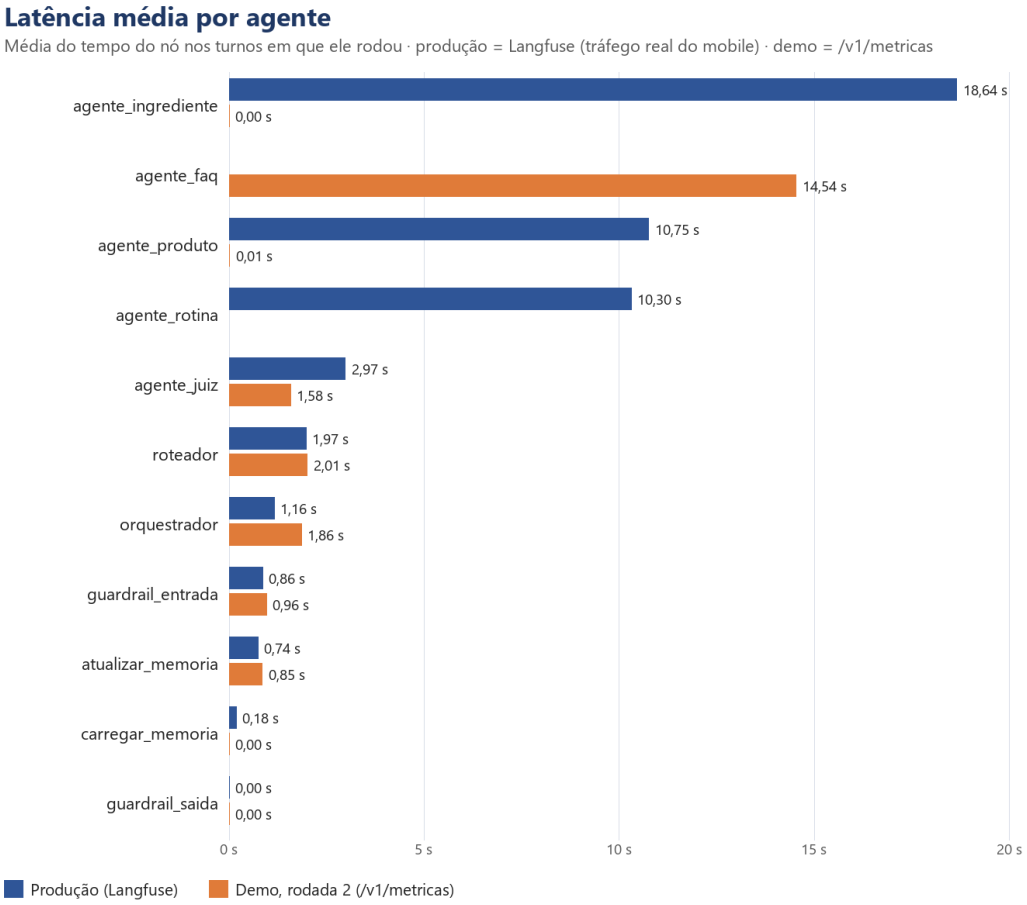
Indicador	Resultado	Tipo
Turnos / usuários distintos	139 / 7	Medido (produção)
Erro de turno (raiz do trace em ERROR)	0 de 139 (0%)	Medido (produção)
Latência total: média, p50, p95, máximo	9,90 s, 5,19 s, 29,29 s, 45,28 s	Medido (produção)
Tempo entre os agentes (fora dos nós)	0,57 s por turno (13,7% do total)	Medido (produção)
Tokens de entrada / saída	2.138.707 / 54.352	Medido (produção)
Tokens por turno (entrada / saída)	15.386 / 391	Medido (produção)
Custo no preço único de US$ 0,20 por 1M tokens	US$ 0,4386 no total; US$ 0,003155 por turno	Estimado
Custo com o preço de cada modelo	US$ 0,4284 (2,3% abaixo do preço único)	Estimado
Gerações sem tokens, excluindo as que falharam	0 de 993	Medido (produção)

Disponibilidade HTTP (middleware da API, coleção `metricas_requisicoes` de produção, lida só para contagens, 13 a 30/10/2026):

Indicador	Resultado	Tipo
Requisições ao `POST /v1/chat`	143: 139 × 200, 3 × 401, 1 × 422	Medido (produção)
HTTP 5xx	0 (disponibilidade de 100%)	Medido (produção)
Duração HTTP: média, p50, p95, máximo	10,13 s, 6,28 s, 29,86 s, 45,52 s	Medido (produção)

Com 143 requisições e nenhuma falha, o limite superior de 95% para a taxa de 5xx é de cerca de 2,1% (regra dos três). Ou seja, a amostra **não prova** uma disponibilidade de 99%.

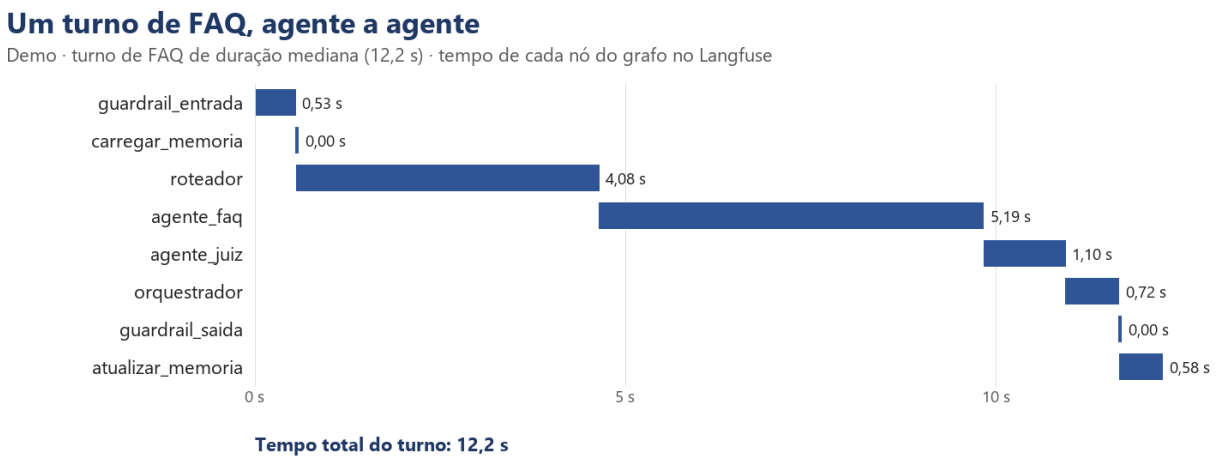
3.2 Latência por agente e entre agentes



Latência média por agente: produção e demo

Agente (nó do grafo)	Execuções (produção)	Média (produção)	Média na demo (rodada 2)
`agente_ingrediente`	1	18,65 s	falha na hora, sem Postgres
`agente_produto`	24	10,75 s	falha na hora, sem Postgres
`agente_rotina`	29	10,30 s	não rodou
`agente_faq`	0	não rodou	14,54 s
`agente_juiz`	54	2,97 s	1,58 s
`roteador`	128	1,97 s	2,01 s
`orquestrador`	54	1,16 s	1,86 s
`guardrail_entrada`	139	0,86 s	0,96 s
`atualizar_memoria`	139	0,74 s	0,85 s
`carregar_memoria`	128	0,18 s	0,00 s
`guardrail_saida`	139	0,004 s	0,003 s

Os especialistas respondem por quase todo o tempo de um turno. O juiz pode reprovar e mandar o especialista rodar de novo, o que dobra o tempo do turno:



Linha do tempo de um turno de FAQ, agente a agente

3.3 Índice de erros

Camada	Produção (139 turnos / 143 requisições)	Demo (84 turnos, rodadas 2 a 4)
HTTP 5xx	0 de 143 (0%)	1 de 84 (1,2%): um 504 por rede degradada
Turno em exceção ou timeout	0 de 139 (0%)	1 de 84 (1,2%)
Falha técnica mascarada (HTTP 200)	não mensurável: o coletor ainda não foi implantado	3 de 84 (3,6%)
Erro do provedor, absorvido pelo fallback	`gpt-oss-120b`: 252 de 347 gerações (72,6%); `ministral-14b-latest`: 3 de 643 (0,5%); `gpt-oss-20b`: 0 de 3	n/d

O erro do provedor primário é o dado mais relevante: o plano gratuito do Groq limita a 8.000 tokens por minuto e devolve 429 em quase três de cada quatro chamadas ao `gpt-oss-120b`. O fallback para o Mistral absorve e o usuário não vê erro, mas cada falha custa tempo.

3.4 Demo controlada com o grafo real

O roteiro (`scripts/observabilidade/demo_grafo_real.py`) tem 28 turnos de 4 usuários `demo-*`: 6 de small talk, 8 de FAQ, 5 fora de escopo, 4 tentativas de prompt-injection e 5 perguntas que o juiz pode reprovar. Roda com LLMs reais e Langfuse ligado; Mongo, Postgres, Qdrant e Firebase de produção ficam zerados (o script se recusa a rodar se algum estiver preenchido) e as coleções de métricas ficam em memória.

Foram feitas quatro rodadas. A rodada 1 usou um roteiro anterior e é só histórico (6 turnos caíram em especialistas de dados e falharam sem Postgres). As rodadas 2, 3 e 4 usam o mesmo roteiro e são agregadas nos resultados (84 turnos).

Rodada	Código	Resolvidos	Erro de turno	Falha técnica	p50	p95	Custo (US$)
1	antes das correções	15 de 28 (53,6%)	0	n/d	2,7 s	28,1 s	0,0506
2	corrigido, flag ainda sem escopo	20 de 28 (71,4%)	0	2 (recalculada)	2,9 s	46,5 s	0,0606
3	final	20 de 28 (71,4%)	1 (504)	0	16,1 s	92,4 s	0,0754
4	final	17 de 28 (60,7%)	0	1	7,1 s	111,3 s	0,0856

Saúde do ambiente em cada rodada (a rede desta máquina oscilou):

Rodada	Falhas de exportação do Langfuse	Conexões resetadas	Busca web falhou	Resposta não 200	Traces perdidos pelo Langfuse
1	0	0	0	0	0 de 28
2	2	0	2	0	0 de 28
3	18	2	2	1	2 de 28
4	18	3	5	0	1 de 28

Leitura: a latência da demo nas rodadas 3 e 4 reflete a rede instável desta máquina e o Groq no plano gratuito, **não** a produção (use a seção 3.1 para latência). A demo serve para cobrir categorias de pergunta e comparar as duas fontes de medição.

Resolução por categoria nas rodadas 2 a 4 (84 turnos):

Categoria	Turnos	Resolvidos	Taxa
Small talk	18	18	100%
Fora de escopo	15	15	100%
Pergunta ao juiz	15	11	73,3%
FAQ	24	13	54,2%
Prompt-injection	12	0	0% (bloqueio correto)
Total	**84**	**57**	**67,9%**

Os 27 turnos não resolvidos: 12 bloqueios corretos do guardrail, 11 reprovações do juiz após as tentativas, 3 falhas técnicas e 1 timeout. Excluindo as 12 tentativas de injection (em que bloquear é o comportamento certo), a taxa sobe para **79,2% (57 de 72)**. As reprovações de FAQ são em parte efeito da demo: sem Qdrant, o FAQ usa o índice local `EmbeddingsHash` (contagem de palavras, sem busca semântica), pior que o de produção.

4. Custo, custo por resolução e ROI

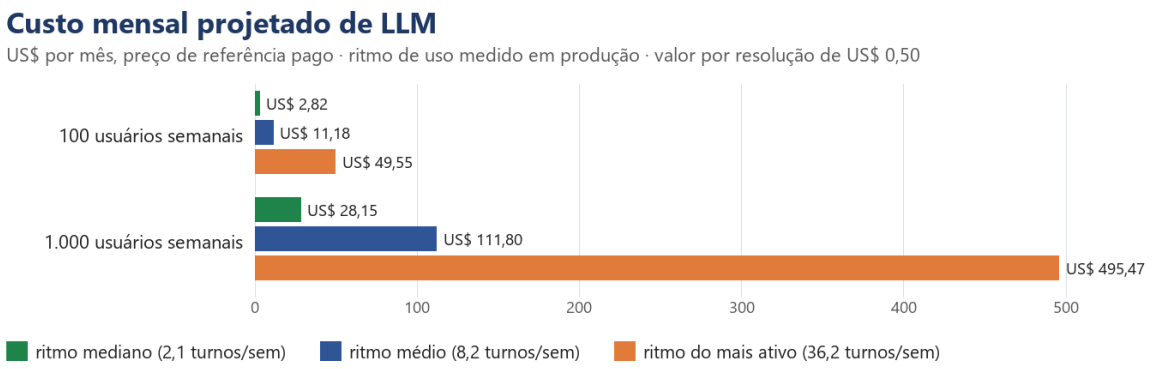
Preço de referência: **US$ 0,20 por 1 milhão de tokens de entrada e de saída** (Ministral 3 14B, que responde por 86% dos tokens de entrada de produção). Custo medido por turno: **US$ 0,003155** (produção, 139 turnos).

Indicador	Valor	Tipo
Taxa de resolução (demo, rodadas 2 a 4)	67,9% (57 de 84)	Medido (demo)
Custo por resolução (todos os turnos)	**US$ 0,00465**	Estimado
Custo por resolução (só turnos legítimos, 79,2%)	US$ 0,00399	Estimado
Ponto de equilíbrio do ROI	US$ 0,00465 por resolução	Calculado
ROI com US$ 0,50 por resolução (conservador)	**106,5×**	Extrapolado
ROI com US$ 1,84 por resolução (autoatendimento)	394,7×	Extrapolado
ROI com US$ 13,50 por resolução (atendimento assistido)	2.902×	Extrapolado

O ROI é enorme porque o custo de inferência por turno (menos de meio centavo) é pequeno diante de qualquer valor razoável de uma resolução. Isso não é retorno financeiro medido: depende só do valor assumido, e o custo de infraestrutura e de manutenção não entra na conta.

5. Projeção para 100 e 1.000 usuários semanais

Hipóteses (todas extrapolação): custo por turno medido em produção (US$ 0,003155); taxa de resolução de 67,9% (demo); custo linear com o número de usuários; o ritmo de uso vem da **distribuição medida** em produção, em turnos por usuário por semana: mediana de 2,1, média de 8,2 e usuário mais ativo de 36,2. A distribuição é muito assimétrica (um usuário faz 63% dos turnos), por isso a faixa é larga.



Custo mensal projetado de LLM

Cenário principal (ritmo médio, 8,2 turnos por usuário por semana):

Cenário	Turnos/semana	Custo/semana	Custo/mês	Resoluções/semana	ROI (US$ 0,50)
100 usuários semanais	818	US$ 2,58	US$ 11,18	555	106,5×
1.000 usuários semanais	8.176	US$ 25,80	US$ 111,80	5.548	106,5×

Faixa conforme o ritmo de uso:

Ritmo medido	Turnos por usuário/semana	100 usuários: custo/mês	1.000 usuários: custo/mês
Usuário mediano	2,1	US$ 2,82	US$ 28,15
Usuário médio	8,2	US$ 11,18	US$ 111,80
Usuário mais ativo	36,2	US$ 49,55	US$ 495,47

Sensibilidade do ROI ao valor por resolução (custo por turno de US$ 0,003155 e 67,9% de resolução):

Valor por resolução	ROI	Base
US$ 0,00465	0×	ponto de equilíbrio
US$ 0,50	106,5×	conservador (padrão do sistema)
US$ 1,84	394,7×	mediana Gartner, contato por autoatendimento
US$ 13,50	2.902×	mediana Gartner, contato em canal assistido

Custos que **não** estão nesta projeção: infraestrutura fixa (ECS, MongoDB, Postgres, Qdrant), domínio, monitoramento e horas de manutenção. Em escala pequena eles tendem a pesar mais que os tokens; precisam ser somados com os valores reais da implantação.

6. Premissas justificadas

6.1 Preço por token

Preços do plano pago de cada provedor, lidos nas páginas oficiais em **30/10/2026** (USD por 1 milhão de tokens):

Modelo nos traces	Equivalente pago	Entrada	Saída	Fonte
`ministral-14b-latest`	Ministral 3 14B (Mistral)	0,20	0,20	docs.mistral.ai/inference/pricing (a página não traz data)
`openai/gpt-oss-120b`	GPT-OSS 120B (Groq)	0,15	0,60	console.groq.com/docs/model/openai/gpt-oss-120b
`openai/gpt-oss-20b`	GPT-OSS 20B (Groq)	0,075	0,30	console.groq.com/docs/model/openai/gpt-oss-20b

Modelo nos traces	Equivalente pago	Entrada	Saída	Fonte
`gemini-3.6-flash`	Gemini 3.6 Flash (Google)	0,75	3,75	ai.google.dev/gemini-api/docs/pricing (atualizada em 28/10/2026; o preço sobe para 1,50 e 7,50 em 01/01/2027; valor lido por uma ferramenta de consulta, confira na página antes de apresentar)

O sistema usa **um preço único de US$ 0,20 por 1M tokens** (`CUSTO_USD_1M_TOKENS_ENTRADA` e `CUSTO_USD_1M_TOKENS_SAIDA`). Com o tráfego real, o preço de cada modelo daria um custo 2,3% menor; a diferença é pequena porque o Ministral responde por 86% dos tokens de entrada e o Gemini não apareceu em produção. O custo que o próprio Langfuse calcula (US$ 0,0499) cobre só os modelos com preço cadastrado: bate exatamente com o custo dos dois modelos do Groq pela tabela acima (US$ 0,0499), o que valida os preços do Groq, mas mostra que o painel **ignora o custo do Ministral** e subestima o total em cerca de 9 vezes. O alias `ministral-14b-latest` não é uma versão fixada, e a equivalência de qualidade com o modelo pago não foi avaliada. Tokens lidos do cache de prompt (9.984 em produção, 0,5% da entrada) foram contados como entrada normal, o que é conservador.

6.2 Valor por resolução

O valor de uma resolução é o que um turno resolvido economiza. Três cenários, aprovados em 30/10/2026:

Cenário	Valor	Base
Conservador (padrão: `VALOR_USD_POR_RESOLUCAO`)	US$ 0,50	Cerca de um quarto da mediana de um contato por autoatendimento (US$ 1,84), supondo uns 4 turnos por contato. **A suposição de 4 turnos é do grupo, não foi medida.**
Referência de autoatendimento	US$ 1,84	Mediana do custo por contato em autoatendimento. Gartner, *Benchmarks to Assess Your Customer Service Costs*, 01/02/2024 (resumo público; o relatório completo é pago)
Referência de atendimento assistido	US$ 13,50	Mediana do custo por contato em canais assistidos (telefone, chat, e-mail), mesma fonte

Um contato de atendimento pode ter vários turnos e o Venus é um app de consumo, não um canal de suporte; por isso o relatório usa o valor conservador como número principal e mostra os outros dois como faixa.

6.3 Definição de resolução e limites

Um turno é "resolvido" quando termina sem erro, com resposta, sem reprovação do juiz e sem bloqueio de guardrail. Limites:

- **Não mede satisfação** nem se o problema foi resolvido no mundo real.
- **Pergunta fora de escopo respondida com um redirecionamento conta como resolvida** (15 de 15 na demo), embora o usuário não tenha obtido o que pediu.
- **Bloqueio correto de prompt-injection conta como não resolvido**, o que reduz a taxa de modo injusto (por isso a taxa dos turnos legítimos é mostrada à parte).
- **Falha técnica não é resolução:** quando o especialista cai, o juiz marca o turno como reprovado e o orquestrador responde um pedido de desculpas; esse turno conta como não resolvido e como falha técnica.
- **Em produção a resolução não é mensurável** sem ler o conteúdo das conversas (não lido, por privacidade) e sem o coletor implantado; por isso a taxa vem da demo.

7. SLOs e SRE

Metas propostas para o piloto, definidas por experiência de uso de um chat. Cada medida usa a melhor fonte disponível.

SLO	Meta	Medido	Situação	Ação sugerida
Disponibilidade do `/v1/chat` (1 − 5xx)	≥ 99,0%	100% (0 de 143, produção)	Atingido na amostra; limite superior de 95% de ~2,1% de 5xx	Repetir a medição com 7 dias de tráfego depois do deploy
Latência p50 do turno	< 8 s	5,2 s (Langfuse) e 6,3 s (HTTP)	Atingido	Manter

SLO	Meta	Medido	Situação	Ação sugerida
Latência p95 do turno	< 20 s	29,3 s (Langfuse) e 29,9 s (HTTP)	**Não atingido**	Menos tokens por turno (15,4 mil de entrada), limite de tentativas do juiz, provedor primário sem 429
Erro de turno (exceção, 5xx, 504)	< 1%	0 de 139 em produção; 1 de 84 na demo	Atingido em produção	Alerta em 504 e 502
Falha técnica mascarada (HTTP 200)	< 2%	produção: não medida; demo: 3 de 84 (3,6%)	**Não atingido na demo; não medido em produção**	Implantar o coletor e acompanhar `falhas_tecnicas`
Resolução dos turnos legítimos	≥ 80%	79,2% (57 de 72, demo)	**Não atingido por 0,8 ponto**	Qdrant com busca semântica no FAQ; revisar reprovações do juiz
Custo por resolução	< US$ 0,01	US$ 0,00465	Atingido	Reavaliar se o modelo ou os preços mudarem
Erro do provedor primário (especialista)	< 10%	`gpt-oss-120b`: 72,6%	**Não atingido**	Plano pago do Groq ou reordenar a cadeia de fallback
Perda de traces no Langfuse	< 1%	0% na rodada 2; 7% (rodada 3) e 3,6% (rodada 4)	**Não atingido com rede degradada**	Manter o coletor da API como fonte independente; buffer maior no exportador

Resumo: **quatro metas atendidas** (disponibilidade na amostra, p50, erro de turno em produção e custo por resolução) e **cinco não atendidas ou não medidas** (p95, falha técnica, resolução dos turnos legítimos, erro do provedor primário e perda de traces).

8. Cruzamento `/v1/metricas` × Langfuse × middleware

Coletor da API × Langfuse (rodada 2, 28 turnos casados um a um):

Medida	Coletor da API	Langfuse	Diferença
Tempo total do turno: mediana da diferença entre as duas fontes	n/a	n/a	1,2 ms
Tempo total do turno: maior diferença (1º turno da rodada)	13,50 s	11,06 s	2,44 s: partida a frio que o cronômetro da API vê e o span raiz do Langfuse não
Tempo médio de `agente_faq`	14,5367 s	14,5369 s	0,2 ms
Tokens de entrada	296.755	296.755	0
Tokens de saída	6.139	6.139	0
Turnos com tokens iguais	28 de 28	28 de 28	0

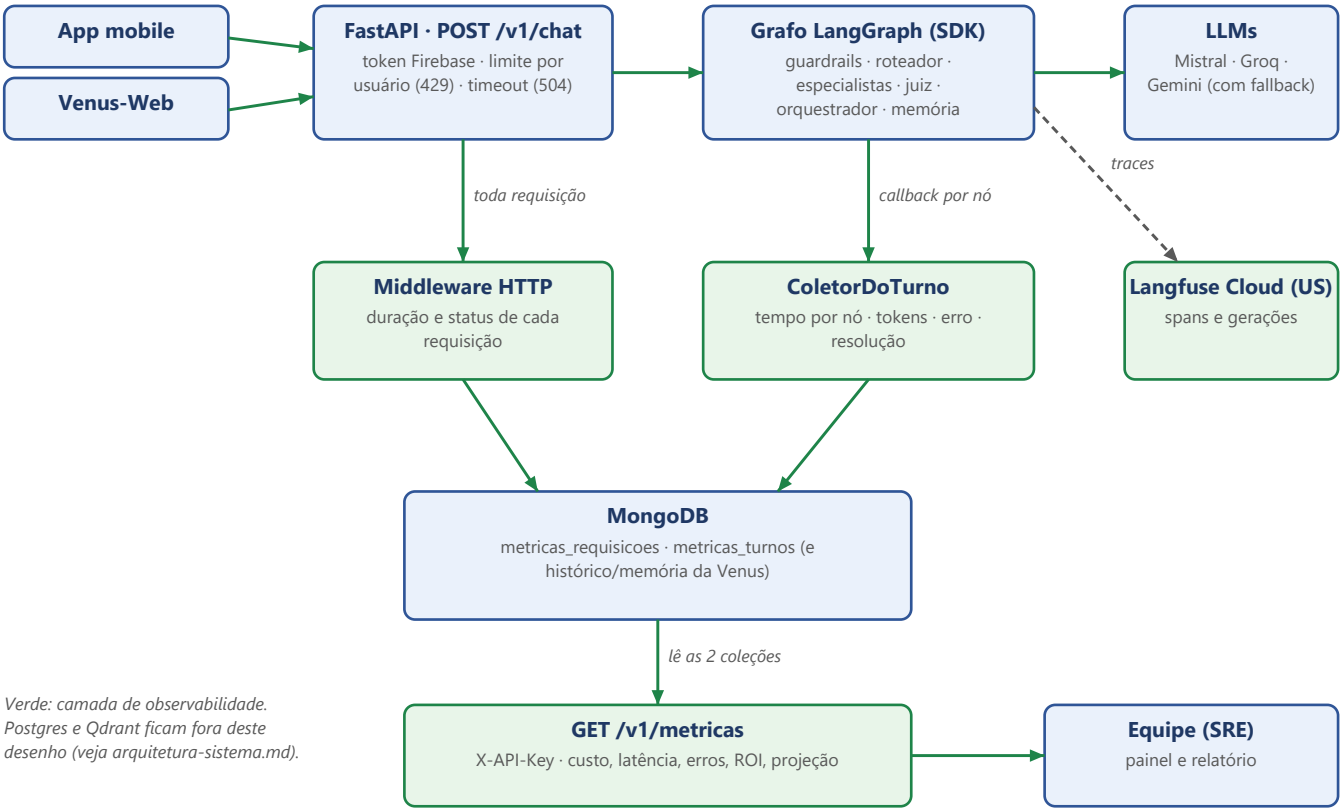
Os tempos por agente coincidem em milissegundos porque os dois medem o mesmo nó do grafo. Os tokens só batem depois de somar ao `input` do Langfuse os tokens lidos do cache de prompt (`input_cache_read`): sem isso o Langfuse mostra 4,2% a menos de entrada (12.544 tokens, 7 vezes 1.792) em 5 dos 28 turnos. O `usage_metadata` do LangChain já soma o cache.

Coletor × Langfuse nas rodadas 3 e 4: o coletor registrou 28 de 28 turnos; o Langfuse tem 26 e 27 traces (o log mostra 18 falhas de exportação de spans por rodada). Sem o coletor, o relatório de erros e custos teria 3,6% a 7% de turnos faltando. As quatro rodadas estão em `docs/evidencias/dados/cruzamento-rodada-N.json`.

Middleware × Langfuse (produção): 143 requisições, menos 4 respostas 4xx, dão 139 respostas 200, o mesmo número de traces reais no Langfuse. O p95 de 29,86 s (HTTP) contra 29,29 s (Langfuse) e o p50 de 6,28 s contra 5,19 s mostram o que o Langfuse não vê: validação do token, busca do usuário no Postgres e serialização, cerca de 1 s por turno.

9. Arquitetura de alto nível

A versão em Mermaid, com todos os componentes (Postgres, Qdrant, MCP, A2A, Google Calendar), está em arquitetura-sistema.md. O desenho abaixo é o de alto nível, com a camada de observabilidade em verde: middleware HTTP → Mongo, coletor de turnos (callback por nó) → Mongo, Langfuse recebendo os traces do grafo e o endpoint `GET /v1/metricas` lendo as duas coleções.

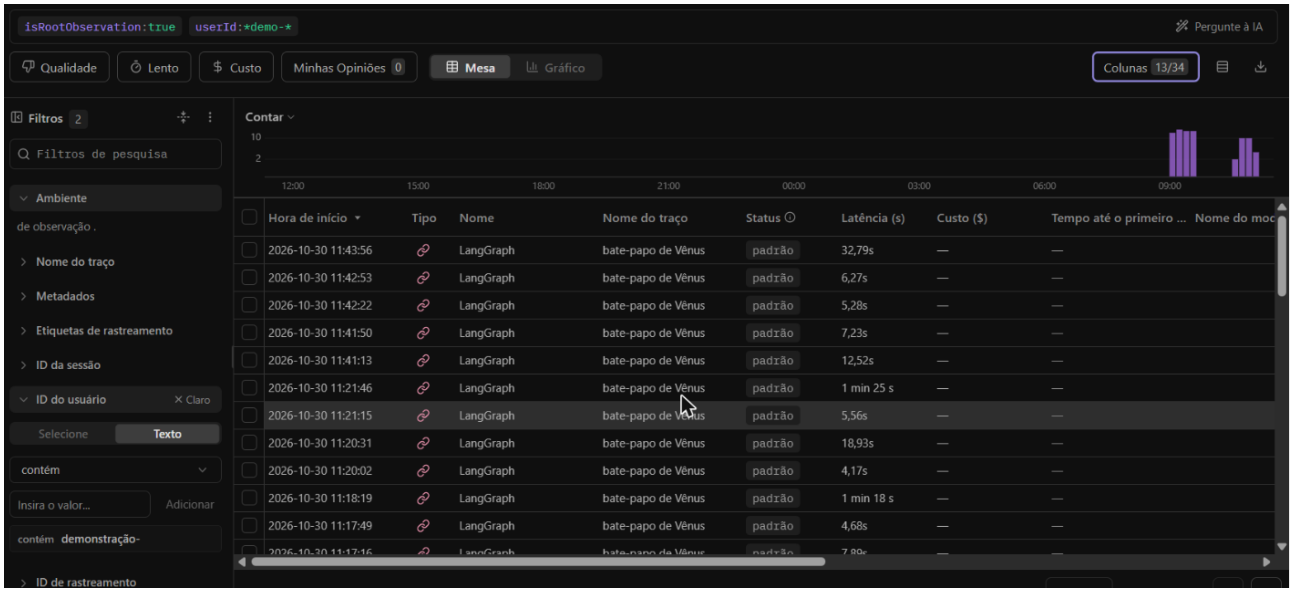


Verde: camada de observabilidade. Postgres e Qdrant ficam fora deste desenho (veja arquitetura-sistema.md).

Arquitetura de alto nível com a camada de observabilidade.

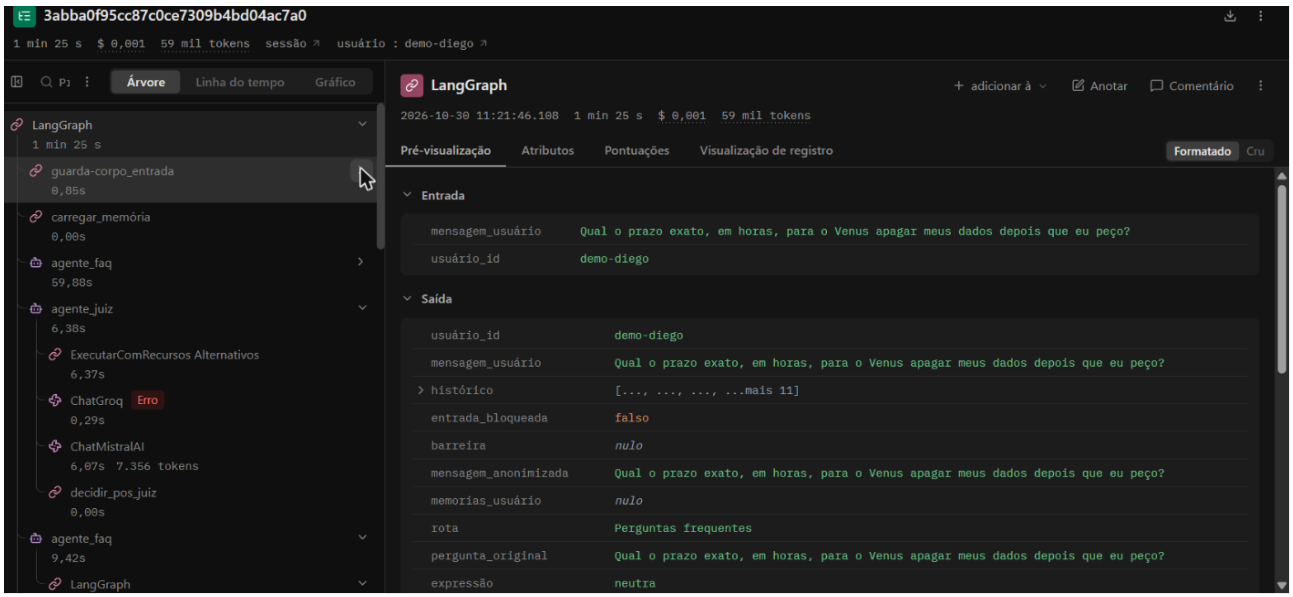
10. Evidências do Langfuse

Capturadas em 30/10/2026 no projeto Langfuse do grupo (região US), sem dados de usuários reais: os prints mostram só usuários `demo-*`, e o widget de "latência máxima por usuário" do dashboard, que lista IDs reais, ficou de fora do recorte.



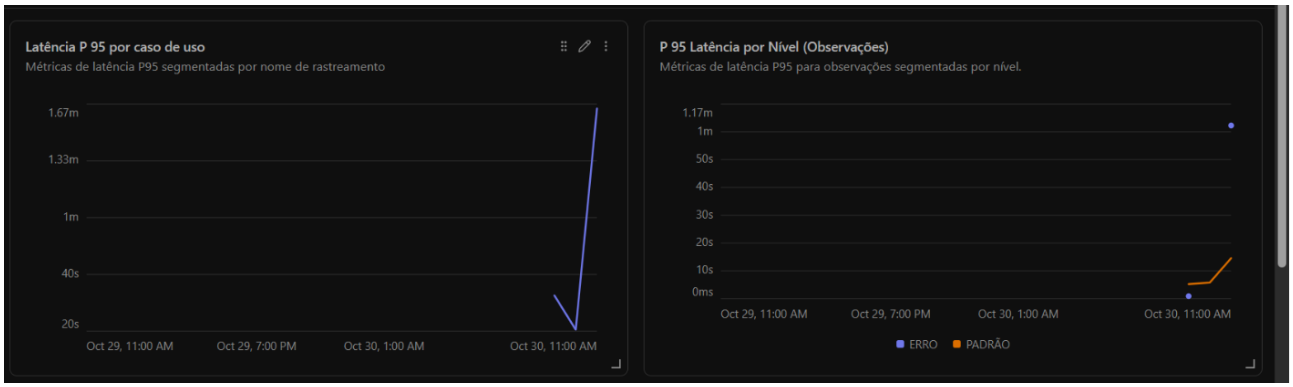
Lista de traces filtrada por userId contendo demo-

Lista de traces filtrada por `userId` contendo `demo-` (cerca de 85 traces nas últimas 24 h, na hora da captura).



Árvore de um turno de FAQ com o tempo de cada agente

Um turno de FAQ da rodada 3 (rede degradada), 1 min 25 s: `guardrail_entrada` 0,85 s; `agente_faq` 59,88 s; `agente_juiz` 6,38 s com a chamada ao `ChatGroq` em **Erro** e o fallback para o `ChatMistralAI` (6,07 s, 7.356 tokens); e uma segunda execução do `agente_faq` (9,42 s).



Dashboard de latência do Langfuse

P95 de latência por caso de uso e por nível (ERRO × PADRÃO) nas últimas 24 h. O pico do fim do gráfico vem das rodadas 3 e 4 da demo com rede degradada.

11. Como reproduzir cada número

Pré-requisitos: Python 3.13 do `.venv` da API (para as extrações), Python 3.14 do sistema com `reportlab`, `pillow` e `pymupdf` (para gráficos e PDF) e o SDK com o conserto do bug do 502 aplicado (`docs/evidencias/sdk-fix-502-gemini.patch`) no `PYTHONPATH`. Os comandos partem da raiz do repositório; `..\.env` é o `.env` da API.

```
# 1. Testes da API (suíte completa, sem tocar em banco algum)
PYTHONPATH="<SDK>/src;." QDRANT_URL= MONGODB_URL= DATABASE_URL= python -m pytest venus_api/tests -q

# 2. Produção, Langfuse (somente leitura, sem conteúdo de conversa)
PYTHONPATH=. python scripts/observabilidade/langfuse_extrair.py --dias 45 --env ../.env --saida-dir saida/producao

# 3. Produção, HTTP (somente leitura, contagens)
PYTHONPATH=. python scripts/observabilidade/mongo_http_leitura.py --env ../.env \
    --desde 2026-10-13T00:00:00+00:00 --ate 2026-10-30T14:58:02+00:00 --saida saida/producao/http.json

# 4. Demo com o grafo real (uma rodada, cerca de 15 a 25 minutos)
PYTHONPATH="<SDK>/src;." python scripts/observabilidade/demo_grafo_real.py --env ../.env \
    --saida-dir saida/demo --pausa 25

# 5. Cruzamento coletor x Langfuse (use o início e o fim da rodada)
PYTHONPATH=. python scripts/observabilidade/cruzar_metricas.py --turnos saida/demo/metricas_turnos_demo.json \
```

```
    --traces saida/producao/langfuse_traces.json --desde <inicio> --ate <fim> --saida saida/cruzamento.json

# 6. Custo, projeção e ROI (a taxa de resolução soma as rodadas informadas)
PYTHONPATH=. python scripts/observabilidade/numeros_do_relatorio.py \
    --langfuse-traces saida/producao/langfuse_traces.json \
    --metricas-demo saida/demo2/metricas_demo.json saida/demo3/metricas_demo.json saida/demo4/metricas_demo.json \
    --saida saida/numeros.json

# 7. Gráficos e PDF
python scripts/observabilidade/graficos.py --langfuse-resumo saida/producao/langfuse_resumo.json \
    --langfuse-traces saida/producao/langfuse_traces.json --metricas-demo saida/demo2/metricas_demo.json \
    --numeros saida/numeros.json --desde <inicio> --ate <fim> --saida-dir docs/evidencias
python scripts/observabilidade/gerar_pdf.py docs/observabilidade.md docs/observabilidade.pdf
```

Número do relatório	Comando	Dado guardado em `docs/evidencias/dados/`
Latência, tokens, erros e provedores de produção	2	`producao-langfuse-resumo.json`, `producao-langfuse-traces.json`
Disponibilidade e latência HTTP de produção	3	`producao-http.json`
Resolução, falha técnica, `/v1/metricas` da demo	4	`demo-rodada-N-metricas.json`, `-turnos-do-coletor.json`, `-respostas.jsonl`
Diferenças entre coletor e Langfuse	5	`cruzamento-rodada-N.json`
Custo, custo por resolução, ROI, projeção	6	`numeros-do-relatorio.json`

Como uma rodada da demo não se repete bit a bit (os LLMs não são determinísticos), os dados das quatro rodadas ficam guardados em `docs/evidencias/dados/`. A fotografia de produção também: uma nova extração terá mais turnos que os 139 usados aqui.

12. Limitações

- **Amostra pequena e enviesada:** 7 usuários, 17 dias, um usuário responde por 63% dos turnos. O ritmo de uso por usuário (2,1 a 36,2 turnos por semana) é o maior fator de incerteza da projeção.
- **Demo sem Postgres e sem Qdrant** (regra de não usar banco de produção): os especialistas de produto, ingrediente e rotina não têm dados e o FAQ usa busca por palavras. A resolução do FAQ (54%) está subestimada em relação à produção.
- **Rede instável nesta máquina** nas rodadas 3 e 4: latências da demo, 504 e perda de traces são, em parte, do ambiente de teste. A latência confiável é a de produção.
- **Os provedores reais são gratuitos** (custo real de US$ 0); o custo é o de um preço pago de referência. O alias do modelo não é uma versão fixada.
- **Infraestrutura fora da conta**, e o cache de prompt não reduz o custo estimado (conservador).
- **Projeção linear:** sem efeito de crescimento de contexto, limites de taxa dos provedores ou degradação sob carga. 100 usuários simultâneos não foram testados.
- **Langfuse:** o custo do painel ignora modelos sem preço cadastrado (o Ministral); spans podem se perder com rede ruim (3 de 84 turnos de demo).
- **Resolução** não mede satisfação nem correção factual e não é mensurável em produção até o coletor ser implantado.
- **Falha técnica** só é detectada em produção depois do deploy; até lá o único sinal é o erro do provedor nos traces.
- **Cruzamento por ordem de chegada** vale só para a demo, em que cada usuário conversa um turno por vez.
- O Mongo de produção foi lido somente para contagens de `metricas_requisicoes`, com autorização explícita; nada foi gravado em banco de produção.

13. Validação automatizada

- **API:** suíte completa com **227 testes aprovados** e nenhuma falha (`QDRANT_URL`, `MONGODB_URL` e `DATABASE_URL` vazios só no processo de teste); os 54 testes de observabilidade, chat e health passam à parte. Os testes novos cobrem o coletor (incluindo LangGraph real com subgrafo), a falha técnica sem vazamento entre turnos, o 504 gravado como erro, a agregação, os scripts desta pasta e a trava que impede qualquer operação de escrita no script de leitura do Mongo.

- **SDK:** os três testes de regressão do bug do 502 (juiz, roteador e memória) falham no `origin/develop` com `AttributeError: 'list' object has no attribute 'strip'` e passam com o conserto; com ele, `tests/nodes`, `tests/guardrails` e `tests/memory` têm 318 testes aprovados e a suíte completa do SDK tem 771 aprovados, 1 pulado e nenhuma falha.

14. Pendências que só o aluno resolve

- Implantar a API no ECS com a rota `/v1/metricas`, `HEALTH_API_KEY` e, se quiser mudar, `CUSTO_USD_1M_TOKENS_ENTRADA`, `CUSTO_USD_1M_TOKENS_SAIDA` e `VALOR_USD_POR_RESOLUCAO`; depois repetir a medição com `GET /v1/metricas?dias=7`.
- Publicar o conserto do SDK (`sdk-fix-502-gemini.patch`) em uma branch própria a partir do `origin/develop` e garantir que a imagem da API o instale.
- Criar a branch e o PR com nome sem o prefixo da ferramenta, em português; nenhum commit foi feito.
- Decidir se apaga os traces `demo-*` do Langfuse (131 traces de demonstração, incluindo as rodadas de ensaio); nada foi apagado.
- Preencher o custo de infraestrutura real (ECS, bancos) para completar a estimativa de custo.

Apêndice. Piloto de 19/10/2026

A primeira versão deste relatório, de 19/10/2026, foi feita com 2 traces reais de 1 usuário em 1 dia (latências de 3,89 s e 6,11 s, 11.459 tokens). Era uma amostra mínima e não cobria erros HTTP, a demo nem os cruzamentos; foi substituída pelas medições acima.